

BILLAR RANKING API

Informe Técnico del Proyecto Integrador
Desarrollo de Software

Autor	Miguel Angel Perez Vargas
Curso	Desarrollo de Software — @sigmotoa
URL Despliegue	https://billar-ranking-api.onrender.com
Repositorio	https://github.com/maperez98/billar-ranking-api
Tecnología Principal	FastAPI + Python 3.12
Base de Datos	PostgreSQL — Neon Cloud
Storage	Supabase Storage
Deploy	Render (CI/CD automático desde GitHub)
Fecha	Junio 2026

1. Objetivo del Proyecto

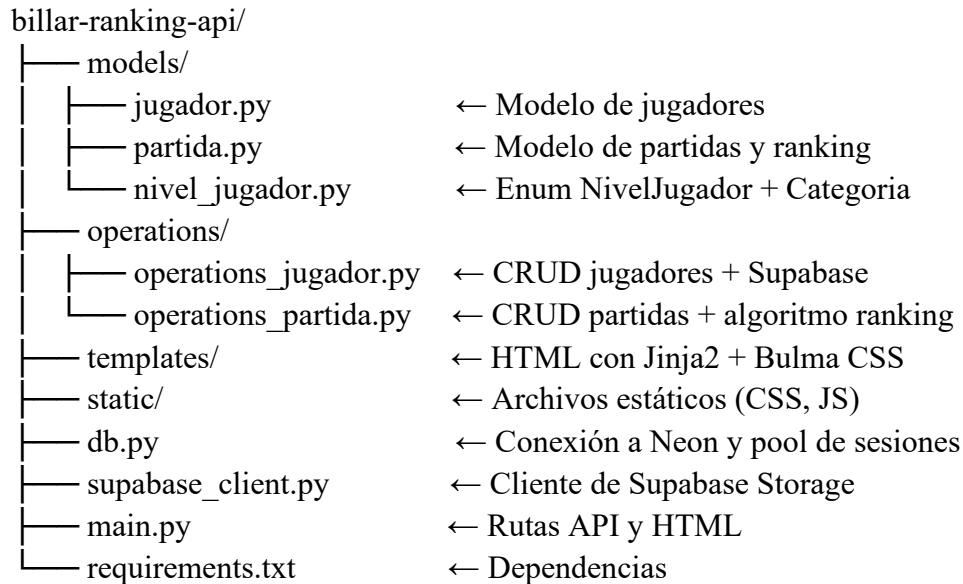
Desarrollar una aplicación web con FastAPI que permita gestionar jugadores de billar, registrar partidas entre ellos, calcular un ranking automático con criterios de desempate claros, y visualizar estadísticas del sistema mediante un dashboard interactivo accesible desde cualquier dispositivo con conexión a Internet.

2. Tecnologías Utilizadas

Categoría	Tecnología	Uso
Backend	FastAPI	Framework principal del servidor web
ORM	SQLModel	Modelos de datos y mapeo objeto-relacional
Base de Datos	Neon (PostgreSQL)	Almacenamiento persistente en la nube
Storage	Supabase Storage	Almacenamiento de fotos de jugadores
Deploy	Render	Hosting con CI/CD automático desde GitHub
Frontend	Jinja2 + Bulma CSS	Plantillas HTML y estilos del sitio
Gráficas	Chart.js	Dashboard con estadísticas visuales
Lenguaje	Python 3.12	Lenguaje de programación principal

3. Arquitectura del Proyecto

3.1 Estructura de Carpetas



3.2 Patrón de Diseño

El proyecto implementa el patrón MVC (Modelo – Vista - Controlador) adaptado al ecosistema FastAPI:

- Modelos (`models/`): Entidades de la base de datos con `SQLModel`, separadas en clases base, de tabla y de actualización parcial.
- Servicios/Operaciones (`operations/`): Lógica de negocio desacoplada de las rutas, organizada por dominio (jugadores y partidas).
- Vistas (`templates/`): Plantillas Jinja2 con Bulma CSS que renderizan HTML dinámico con los datos del servidor.
- Controlador (`main.py`): Define todos los endpoints HTTP que conectan rutas con la capa de operaciones.

4. Capturas de Pantalla

4.1 Página Principal — Ranking de Jugadores

La página de inicio muestra el ranking clasificatorio con el podio de los tres mejores jugadores, sus puntos acumulados y porcentaje de victorias. Incluye navbar con buscador integrado y botón de recálculo del ranking.

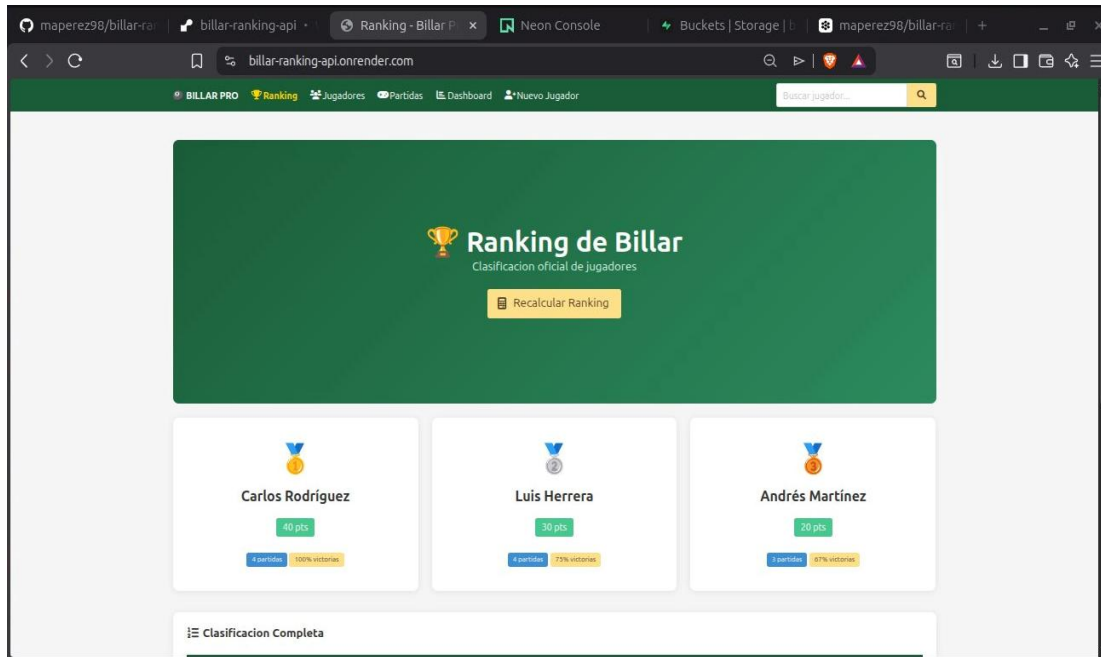


Figura 1. Vista principal — Ranking en billar-ranking-api.onrender.com

5. Modelos de Datos

5.1 Jugador

Campo	Tipo	Descripción / Validación
id	int (PK)	Identificador único, autoincremental
nombre	str	Nombre del jugador — mínimo 2, máximo 50 caracteres
edad	int	Edad válida entre 18 y 100 años
pais	str	País de origen — Predeterminado
nivel	NivelJugador (enum)	principiante / intermedio / profesional
activo	bool	Estado del jugador — True por defecto (soft delete)
foto_url	Optional[str]	URL de foto almacenada en Supabase Storage
categoria_id	Optional[int] FK	Referencia a la tabla Categoria

5.2 Partida

Campo	Tipo	Descripción / Validación
id	int (PK)	Identificador único
jugador1_id	int (FK)	ID del primer jugador
jugador2_id	int (FK)	ID del segundo jugador
ganador_id	int (FK)	Debe ser jugador1_id o jugador2_id
fecha	date	Fecha en que se realizó la partida

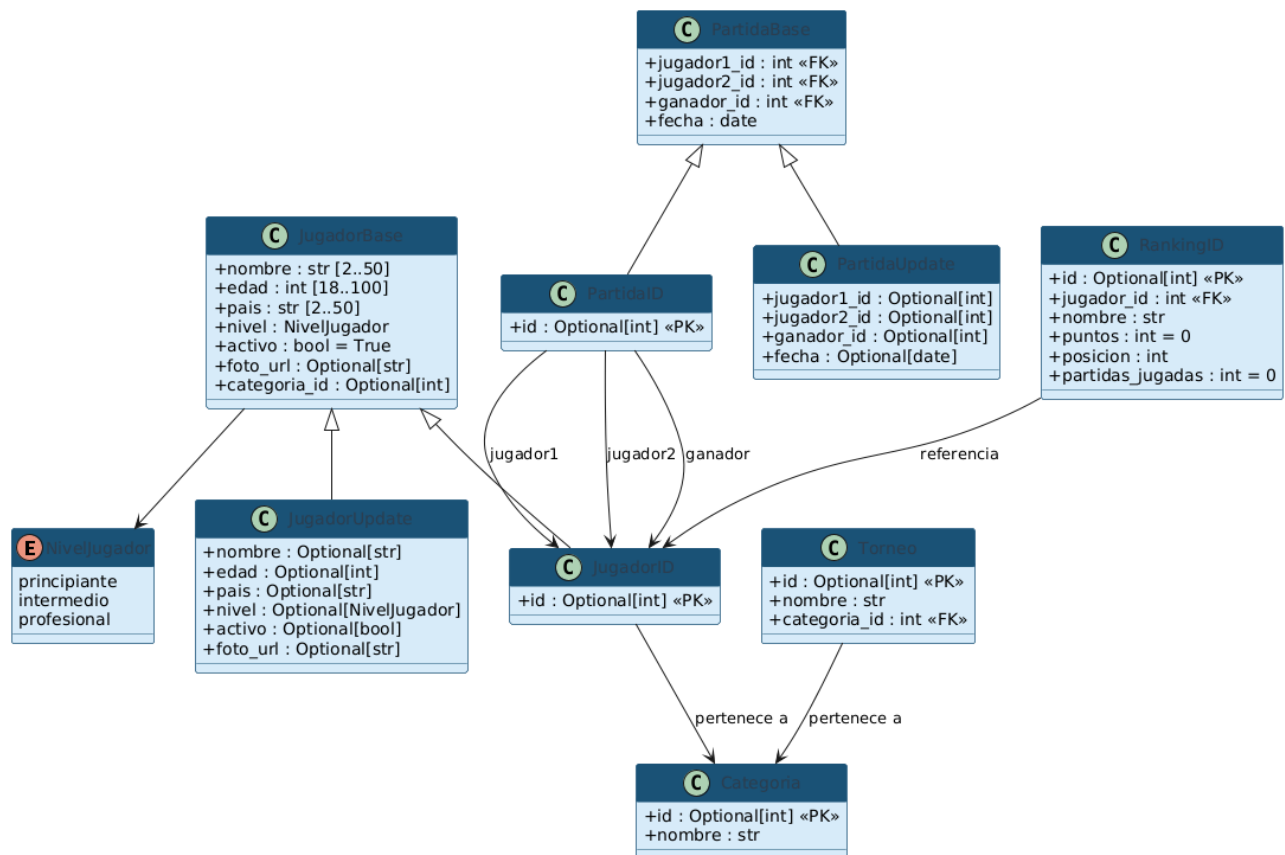
5.3 Ranking

Campo	Tipo	Descripción
id	int (PK)	Identificador único
jugador_id	int (FK)	Referencia al jugador clasificado
nombre	str	Nombre del jugador (desnormalizado)
puntos	int	Puntos acumulados — 10 por cada victoria
posicion	int	Posición en el ranking (1 = líder)
partidas_jugadas	int	Total de partidas del jugador (desempate)

6. Diagramas UML — PlantUML

6.1 Diagrama de Clases

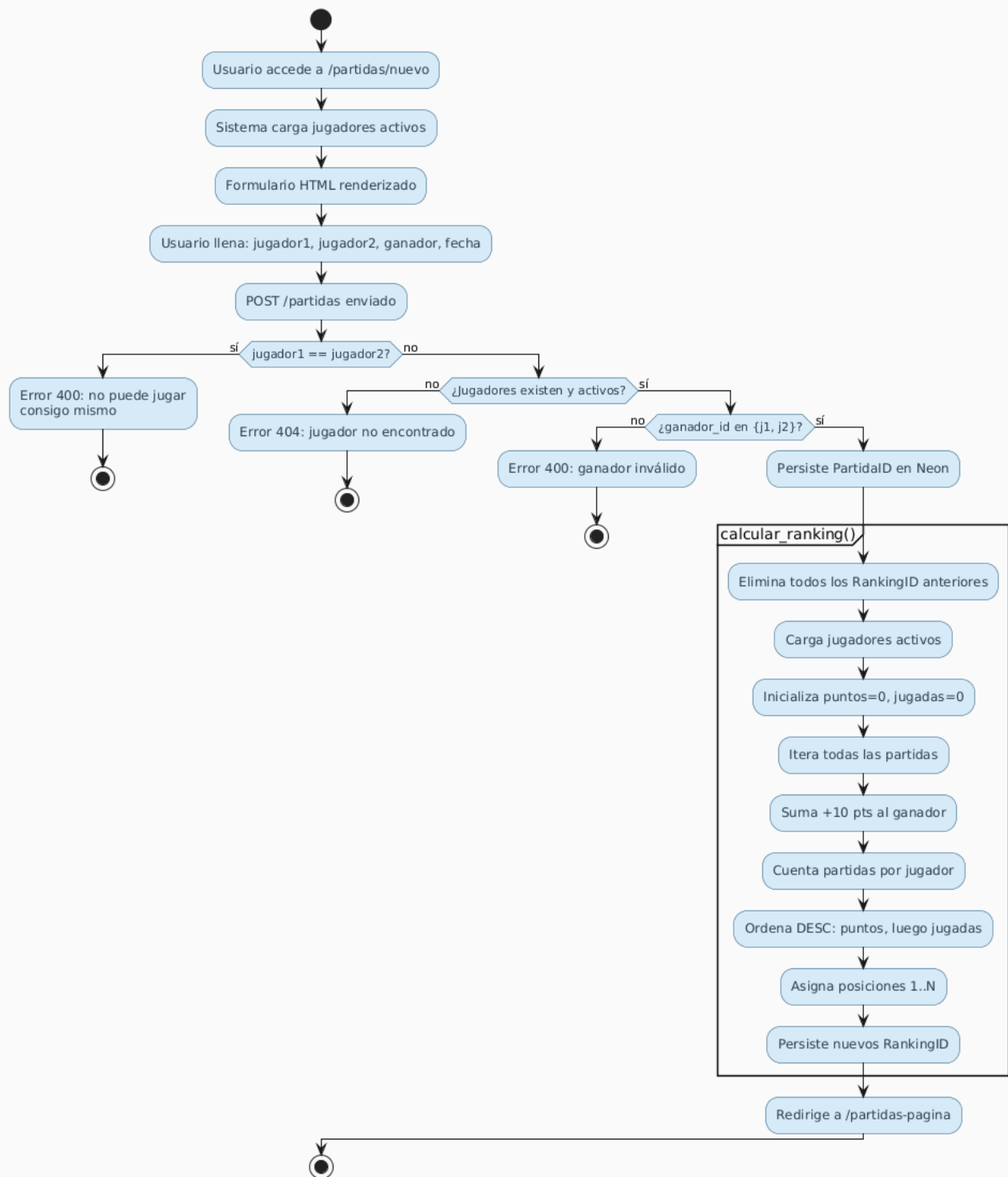
Muestra todas las entidades del sistema, sus atributos, relaciones de herencia (SQLModel) y asociaciones de clave foránea entre tablas.



6.2 Diagrama de Actividades

Flujo completo del registro de una partida, incluyendo todas las validaciones de negocio en el backend y el recálculo automático del ranking.

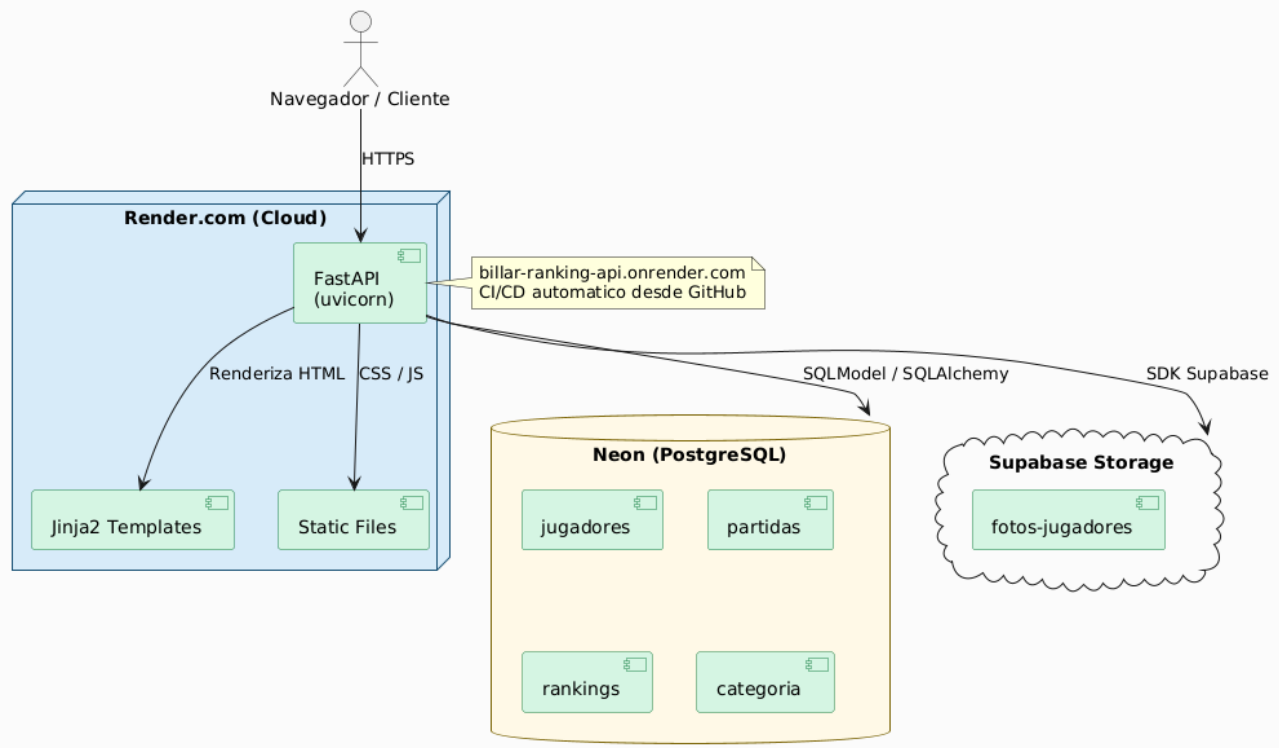
Flujo — Registro de Partida y Recalculo de Ranking



6.3 Diagrama de Despliegue

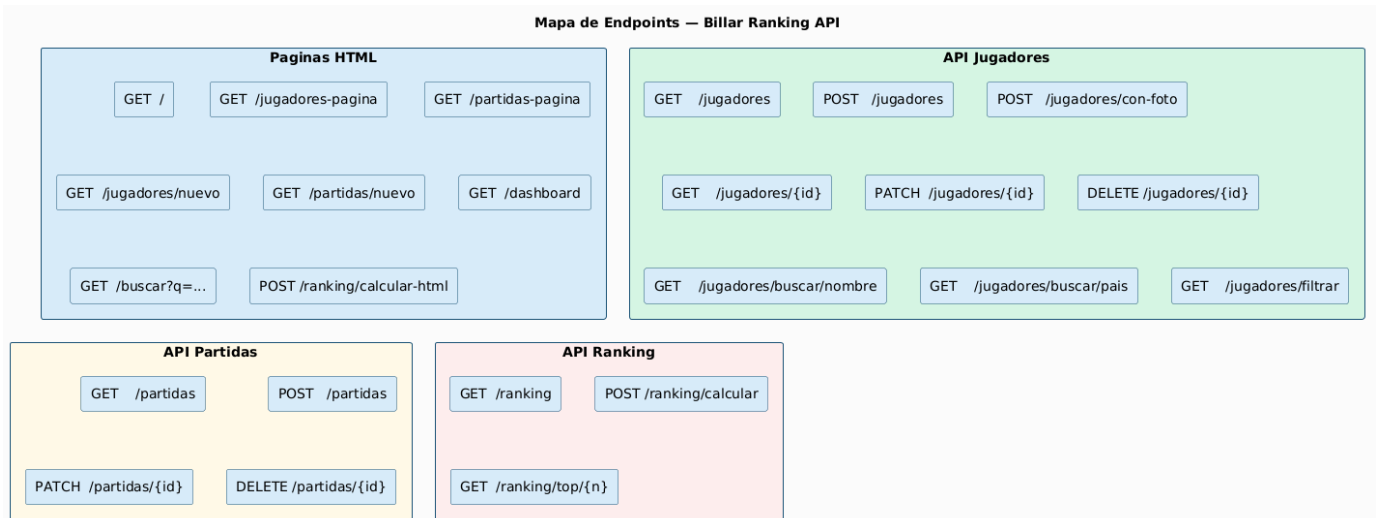
Infraestructura del sistema: servidor FastAPI en Render, base de datos PostgreSQL en Neon Cloud y almacenamiento de imágenes en Supabase Storage.

Arquitectura de Despliegue



6.4 Mapa de Endpoints

Agrupación visual de todos los endpoints por dominio: páginas HTML, API de jugadores, API de partidas y API de ranking.



7. Documentación de Endpoints

7.1 Páginas HTML

Método	Ruta	Descripción
GET	/	Página principal con ranking de jugadores
GET	/jugadores-pagina	Lista completa de jugadores
GET	/partidas-pagina	Lista de todas las partidas registradas
GET	/jugadores/nuevo	Formulario para registrar un jugador
GET	/partidas/nuevo	Formulario para registrar una partida
GET	/dashboard	Dashboard con 4 gráficas estadísticas
GET	/buscar?q=&tipo=	Búsqueda de jugadores por nombre o país
POST	/ranking/calcular-html	Recalcula ranking y re-renderiza la página

7.2 API — Jugadores

Método	Ruta	Descripción
GET	/jugadores	Listar jugadores activos
GET	/jugadores/inactivos	Listar jugadores inactivos
POST	/jugadores	Crear jugador con JSON
POST	/jugadores/con-foto	Crear jugador con foto (multipart/form-data)
GET	/jugadores/{id}	Obtener jugador por ID
PATCH	/jugadores/{id}	Actualizar parcialmente un jugador
DELETE	/jugadores/{id}	Desactivar jugador (soft delete)
GET	/jugadores/buscar/nombre	Buscar por nombre (búsqueda parcial)
GET	/jugadores/buscar/pais	Buscar por país (búsqueda parcial)
GET	/jugadores/filtrar	Filtrar por atributo (pais, nivel, nombre)

7.3 API — Partidas

Método	Ruta	Descripción
GET	/partidas	Listar todas las partidas
POST	/partidas	Registrar nueva partida y recalcular ranking
PATCH	/partidas/{id}	Actualizar datos de una partida
DELETE	/partidas/{id}	Eliminar una partida

7.4 API — Ranking

Método	Ruta	Descripción
GET	/ranking	Ver ranking actual ordenado por posición
POST	/ranking/calcular	Recalcular y persistir ranking completo
GET	/ranking/top/{limite}	Obtener Top N jugadores del ranking

8. Variables de Entorno y Configuración

Crear archivo .env en la raíz del proyecto con:

```
DATABASE_URL_NEON=postgresql://usuario:password@host/neondb?sslmode=require  
SUPABASE_URL=https://tuproyecto.supabase.co  
SUPABASE_KEY=tu_clave_publica
```

8.1 Pool de Conexiones (db.py)

Configuración optimizada para el entorno de producción en Render:

- `pool_size=5`: Máximo de conexiones persistentes activas en el pool.
- `max_overflow=10`: Conexiones adicionales permitidas bajo alta carga.
- `pool_recycle=300`: Recicla conexiones cada 5 minutos para evitar timeouts de Neon.
- `pool_pre_ping=True`: Verifica la salud de la conexión antes de usarla.

9. Ejecución Local

1. Clonar el repositorio

```
git clone https://github.com/maperez98/billar-ranking-api.git  
cd billar-ranking-api
```

2. Crear y activar entorno virtual

```
python -m venv .venv  
source .venv/bin/activate    # Linux / Mac  
.venv\Scripts\activate      # Windows
```

3. Instalar dependencias

```
pip install -r requirements.txt
```

4. Crear archivo .env con las variables de entorno

5. Correr el servidor

```
uvicorn main:app --reload
```

Aplicación: <http://localhost:8000>

Documentación Swagger: <http://localhost:8000/docs>

10. Funcionalidades Implementadas

- CRUD completo de jugadores con validaciones en frontend y backend.
- Subida de foto de jugador a Supabase Storage con URL pública persistente.
- Registro de partidas con validación de jugadores activos, unicidad de participantes y ganador válido.
- Algoritmo de ranking: 10 puntos por victoria, desempate por partidas jugadas.
- Recálculo automático del ranking al registrar cada partida nueva.
- Dashboard interactivo con 4 gráficas (Chart.js): ranking, niveles, victorias y distribución por países.
- Búsqueda de jugadores por nombre y por país desde el navbar con buscador integrado.
- Soft delete de jugadores (campo activo=False, sin borrado físico de la base de datos).
- Manejo global de excepciones con página de error personalizada.
- Despliegue en Render con CI/CD automático conectado al repositorio de GitHub.

11. Autoría del Proyecto

Nombre	Miguel Angel Perez Vargas
Proyecto	Billar Ranking API
Asignatura	Desarrollo de Software
GitHub	https://github.com/maperez98
Repositorio	https://github.com/maperez98/billar-ranking-api
Demo	https://billar-ranking-api.onrender.com
Fecha entrega	Junio 2026